

SWGAN for Image-to-Image Translation

A Project Progress Report
Submitted in partial fulfillment for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
Submitted by

U AJAY SAI	(N190014)
V LAKSHMI PRIYA	(N190914)
M SATYA VINAYAK	(N190260)
P BHARGAVA RAMA KRISHNA REDDY	(N190106)
S BHASKAR RAO	(N190969)

Under the Esteem Guidance of

Mr.KRISHNA KUMAR SINGH
Asst Professor, CSE Dept



**DEPARTMENT OF COMPUTER
SCIENCE AND ENGINEERING**

Rajiv Gandhi University of Knowledge Technologies
Nuzvid, Eluru, Andhra Pradesh – 521202



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies –
Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202

CERTIFICATE OF COMPLETION

This is to certify that the work entitled, “**SWGAN for image-to-image translation**” is the bonafide work of **U. AJAY SAI** (ID No: N190014), **V.LAKSHMI PRIYA** (ID No: N190914), **M. SATYA VINAYAK** (ID No: N190260), **P. BHARGAVA RAMA KRISHNA REDDY** (ID No: N190106), **S. BHASKAR RAO** (ID No: N190969) carried out under our guidance and supervision for the 3rd year project of Bachelor of Technology in the department of Computer Science and Engineering under RGUKT IIIT Nuzvid. This work was done during the academic session July 2024 – April 2025, under our guidance.

Mr.Krishna Kumar Singh
Assistant Professor,
Department of CSE,
RGUKT Nuzvid.

Dr. S. Bhavani
Assistant Professor,
Head of the Department,
Department of CSE,
RGUKT Nuzvid.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies
Nuzvid, Eluru, Andhra Pradesh – 521202

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, “**SWGAN for image-to-image translation**” is the bonafide work of **U. AJAY SAI** (ID No: N190014), **V.LAKSHMI PRIYA** (ID No: N190914), **M. SATYA VINAYAK** (ID No: N190260), **P. BHARGAVA RAMA KRISHNA REDDY** (ID No: N190106), **S. BHASKAR RAO** (ID No: N190969) and hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in final year of Bachelor of Technology for which it has been submitted. This approval does not necessarily endorse or accept every statement made, opinion expressed or conclusion drawn, as a recorded in this thesis. It only signifies the acceptance of this thesis for the purpose for which it has been submitted.

Mr.Krishna Kumar Singh
Assistant Professor,
Department of CSE,
RGUKT-NUZVID

Project Examiner
RGUKT-Nuzvid



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies
Nuzvid, Eluru, Andhra Pradesh – 521202

DECLARATION

“**U. AJAY SAI** (ID No: N190014), **V.LAKSHMI PRIYA** (ID No: N190914), **M. SATYA VINAYAK** (ID No: N190260), **P. BHARGAVA RAMA KRISHNA REDDY** (ID No: N190106), **S. BHASKAR RAO** (ID No: N190969) hereby declare that the project report entitled “**SWGAN for image-to-image translation**” done by us under the guidance of **Mr.Krishna Kumar Singh**, Assistant Professor is submitted for the partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering during the academic session July 2024 - April 2025 at RGUKT-Nuzvid.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references..

Date: 26-04-2025

Place: Nuzvid

U. AJAY SAI	(N190014)
V.LAKSHMI PRIYA	(N190914)
M. SATYA VINAYAK	(N190260)
P. BHARGAVA RAMA KRISHNA REDDY	(N190106)
S. BHASKAR RAO	(N190969)

ACKNOWLEDGEMENT

We would like to express our heartfelt profound gratitude and deep regards to our guide **Mr.Krishna Kumar Singh** for his exemplary guidance, monitoring, and constant encouragement to us throughout the B.Tech course. We shall always cherish the time spent with him during the course of this work due to the invaluable knowledge gained in the field of reliability engineering.

We are extremely grateful for the confidence bestowed in us and entrusting our project entitled “**SWGAN for image-to-image translation**”.

We express gratitude to **Dr. S. Bhavani** (HOD of CSE) and other faculty members for being a source of inspiration and constant encouragement which helped us in completing the project successfully.

Our sincere thanks to all the batch mates of **2019 CSE**, who have made our stay at **RGUKT-NUZVID**, a memorable one.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved God and parents for their blessings, our friends for their help and wishes for the successful completion of this project.

ABSTRACT

In this project, we present SWGAN (Sparse-enhanced Wasserstein Generative Adversarial Network), a novel GAN-based framework for robust and controllable image-to-image translation. The model builds upon the Wasserstein GAN with Gradient Penalty (WGAN-GP) to ensure stable training and improved convergence, and introduces a sparse loss component to enhance feature disentanglement and reduce redundant activations in the generated images.

Our data pipeline is designed using a custom `DataGeneratorRaviLabel` class that constructs few-shot image batches with controlled perturbations and masked blending. This enables the generator to learn meaningful representations by seeing limited but diverse examples. The generator network is a deep convolutional model that outputs both the translated image and intermediate feature activations, while the discriminator evaluates real and fake samples using progressive convolutional layers with normalization and dropout for regularization.

During training, the discriminator is updated multiple times per generator step to maintain the critic’s reliability, and a gradient penalty is applied to enforce Lipschitz continuity. Additionally, when enabled, a sparse loss penalizes over-activation in the generator’s internal layers to encourage more focused and interpretable feature learning.

Extensive visualizations and pixel-wise difference maps are used to analyze the quality and fidelity of translated images. The entire system is optimized using Adam optimizers with carefully tuned hyperparameters, and results show promising generalization for image translation tasks, especially in low-data regimes.

This work demonstrates the effectiveness of combining sparse constraints with WGAN-GP, contributing a more controlled and stable approach to image-to-image translation problems.

Contents

1	Introduction	5
2	Related Works	6
3	Wasserstein Generative Adversarial Network (WGAN)	7
3.1	Purpose	7
3.2	Key Innovation	7
3.3	Advancement	7
3.4	Limitation	7
4	Proposed Approach: SWGAN	8
4.1	Purpose	8
4.2	Innovative Approach	8
4.3	Smoother Convergence	8
4.4	Performance Metrics	8
4.5	Significance	8
5	Algorithm	9
5.1	Step-1: Initialization	9
5.2	Step-2: Data Preprocessing	9
5.3	Step-3: Model Definition	9
5.4	Step-4: Training Functions	9
5.5	Step-5: Training Loop	9
5.6	Step-6: Evaluation	10
5.7	Model Architecture	10
6	Experiments - Datasets Used	11
6.1	Libraries Used	11
6.2	MiniImageNet Dataset:	11
7	Formulas Used :	12
8	Experiments - Comparative Analysis	12
9	System Hardware	12
10	Sample Code	13
11	Output	22
12	SWGAN OUTPUT FOR 100 EPOCH	23
13	SWGAN GRAPH FOR 100 EPOCHS	24
14	Conclusion	25
14.1	Summary of SWGAN's Contributions	25
14.2	Advancements over Traditional GANs and WGANs	25
14.3	Future Potential in Generative Modeling and Data Augmentation	25

15 Future Enhancements	26
15.1 Areas for Future Research and Development	26
15.2 Potential Applications and Extensions of SWGAN	26

1 Introduction

In recent years, the field of Generative Adversarial Networks (GANs) has revolutionized the capabilities of machine learning models, especially in tasks that require generating new data, such as image synthesis, style transfer, and image-to-image translation. Image-to-image translation, which involves transforming an image from one domain to another, has been a core research problem in computer vision. Examples include tasks such as converting black-and-white images to color, transforming sketches into photographs, or even translating between different artistic styles. Despite significant advancements, challenges remain in maintaining the quality, stability, and diversity of generated images.

This thesis introduces the SWGAN (Sparse-Weighted GAN), an advanced framework for image-to-image translation that incorporates sparse loss, controlled perturbations, and Lipschitz continuity into the traditional GAN structure. The SWGAN aims to improve image translation quality by generating high-fidelity images while ensuring stable training through novel regularization techniques and loss functions. The key features of this project are:

Sparse Loss: Encourages the generator to produce more interpretable and disentangled features by penalizing unnecessary activations. This leads to more efficient representation learning and helps reduce the redundancy in the latent space.

Controlled Perturbations: The introduction of perturbations such as noise or blended images during training forces the model to generalize well, rather than overfitting to exact pixel values. This helps improve the robustness and diversity of the generated images.

Lipschitz Continuity: Using a gradient penalty on the discriminator ensures that the model satisfies the Lipschitz constraint, which is crucial for stabilizing the training process in WGANs and mitigating issues like mode collapse.

Regularization via Dropout: Dropout is used in both the discriminator and generator to prevent overfitting and improve generalization by forcing the model to learn more robust and flexible features.

The SWGAN framework is designed to work efficiently on a range of image-to-image translation tasks, leveraging the power of Wasserstein GANs (WGANs) while introducing new techniques to address common challenges faced by traditional GANs. These include the stabilization of training dynamics, ensuring diversity in the generated outputs, and producing visually realistic images with high fidelity.

Through a series of experiments, this thesis demonstrates that SWGAN can effectively learn to translate between different domains of images, yielding superior performance compared to traditional GANs and existing state-of-the-art techniques. Moreover, the integration of sparse loss and other regularization techniques opens new avenues for improving the efficiency and interpretability of generative models.

2 Related Works

Title	Author	Publisher, Year	Key Concept	Cons
GAN: Generative adversarial nets	Goodfellow, Ian, et al	NIPS 2014	The Base line Model	Unguided Class
cGAN: Conditional Generative Adversarial Nets.	Mirza, Mehdi, and Simon Osindero	<i>arXiv, 2014</i>	uses Class label to generate class specific samples	Suffers from vanishing gradients, and Mode Collapse
Metagan: An adversarial approach to few-shot learning	Zhang, Ruixiang, et al	NIPS 2018	uses support set to generates a sample as fake class for boundary smoothing	Insignificant Improvement
ProtoGAN: Towards Few Shot Learning for Action Recognition	Kumar Dwivedi, Sai, et al	CVF 2019	Conditioning a conditional generative adversarial network with class prototype vectors	Class prototypes may not be representative for Few set novel class
Contragan: Contrastive learning for conditional image generation [36]	Kang, Minguk, and Jaesik Park	NIPS 2020	Uses image (batch data-to-data relation) embedding as well as the data-to-class relations by using a Contrastive loss	Some vanishing gradients Still there
LFS-GAN: Lifelong Few-Shot Image Generation	Seo, Juwon, Ji-Su Kang	ICCV, 2023	Task Based Generation, Consider the loss function including the loss of the last task	

3 Wasserstein Generative Adversarial Network (WGAN)

3.1 Purpose

Wasserstein Generative Adversarial Network (WGAN) was introduced to address several inherent limitations of traditional Generative Adversarial Networks (GANs). Traditional GANs often suffer from issues such as mode collapse, where the generator produces limited varieties of outputs, and training instability, where the training process is sensitive and prone to failure. WGAN aims to mitigate these problems and provide a more robust framework for training GANs.

3.2 Key Innovation

The key innovation of WGAN is the utilization of the Wasserstein distance, also known as the Earth Mover's distance, as the loss function. Unlike the Jensen-Shannon divergence used in traditional GANs, the Wasserstein distance provides a smoother and more meaningful measure of the difference between the real and generated data distributions. This allows for a more stable gradient during training, facilitating better convergence and reducing the likelihood of mode collapse.

3.3 Advancement

WGAN offers significant advancements in the training dynamics of GANs. By employing the Wasserstein distance, WGAN stabilizes the training process, allowing for more reliable and consistent progress. This leads to the generation of higher-quality and more realistic synthetic data. The smoother gradient provided by the Wasserstein distance helps avoid the vanishing gradients problem, which is common in traditional GANs and can halt the training process.

3.4 Limitation

Despite the notable improvements introduced by WGAN, it still encounters the mode collapse problem to some extent. Mode collapse is when the generator fails to capture the full diversity of the training data, producing outputs that lack variation. This indicates that while WGAN has made strides in improving GAN training, there remains a need for further research and development to fully overcome this limitation and enhance the robustness and diversity of generated data.

4 Proposed Approach: SWGAN

4.1 Purpose

SWGAN, or Sparse Wasserstein Generative Adversarial Network, was developed to overcome key limitations of traditional GANs, including mode collapse, lack of control in generated features, and training instability. By integrating sparsity regularization and the Wasserstein distance, SWGAN aims to generate more diverse, realistic, and controllable image outputs. This model is particularly effective in tasks involving image-to-image translation, where preserving both structure and variation is crucial.

4.2 Innovative Approach

SWGAN introduces a combination of Wasserstein loss with gradient penalty and a sparse loss that regularizes intermediate activations. The architecture also supports disentangled representations, allowing the generator to better separate and control the various factors of variation within the image data. This promotes a more structured learning process and leads to improved output consistency. Furthermore, the dual-input augmentation strategy—combining perturbed images and mixed-support compositions—enhances the model’s capacity to learn robust mappings..

4.3 Smoother Convergence

SWGAN shows significantly more stable training behavior compared to standard GANs. The use of Wasserstein loss with gradient penalty ensures Lipschitz continuity, leading to reliable gradients for optimization. The integration of sparse loss further encourages the model to focus on essential features, reducing unnecessary fluctuations during training. These innovations together result in smoother convergence, which is critical for training deep generative models efficiently.

4.4 Performance Metrics

Reduced Discriminator Loss: The discriminator in SWGAN becomes more proficient at distinguishing between real and generated images, as shown by a consistently lower discriminator loss. This helps the generator refine its outputs more effectively.

Increased Generator Loss: A higher generator loss signifies that the discriminator is presenting meaningful challenges to the generator. As a result, the generator is pushed to produce more realistic and diverse samples, reducing the risk of mode collapse.

4.5 Significance

SWGAN offers a novel and effective solution to the limitations of traditional GAN architectures. By combining Wasserstein-based training, sparsity constraints, and disentangled representation learning, SWGAN improves the diversity, quality, and control of generated images. These contributions make SWGAN particularly valuable for image translation tasks, and its techniques pave the way for future advancements in generative modeling and regularization strategies.

5 Algorithm

5.1 Step-1: Initialization

The initial step involves setting up the environment and importing all necessary libraries required to implement the SWGAN. This includes configuring GPU/TPU settings, loading the mini-ImageNet dataset, and dividing it into training, validation, and test sets. In this step, the generator and discriminator architectures are also defined using Convolutional Neural Networks (CNNs). Furthermore, custom loss functions such as the Wasserstein loss with gradient penalty and sparse loss are implemented, along with utility functions required for data augmentation and training.

5.2 Step-2: Data Preprocessing

In this phase, input images are normalized to ensure uniformity across the dataset. To improve generalization and robustness, controlled perturbations such as noise masks or blended support images are introduced. A custom data loader class called `Data_GeneratorRaviLabel` is used to efficiently sample, preprocess, and batch data during training. This ensures dynamic pairing of foreground and background image variations, aligning with the goal of learning disentangled representations.

5.3 Step-3: Model Definition

This step focuses on constructing the generator and discriminator models using sequential CNN layers with Layer Normalization and LeakyReLU activations. The generator is designed to not only produce realistic images but also yield intermediate hidden features for sparse regularization. The discriminator is built to output a scalar value indicating real vs. fake, as required by the Wasserstein loss. Both models are designed to ensure Lipschitz continuity, a key requirement for WGAN-GP stability.

5.4 Step-4: Training Functions

In this stage, important components for training are defined. This includes custom distance metrics, the Wasserstein loss with gradient penalty, and the sparse KL-divergence loss. The core `train_step` function performs a forward pass for both generator and discriminator, computes respective losses, calculates gradients, and applies them using optimizers. The sparse loss regularizes the hidden activations, encouraging the model to focus on essential features while ignoring redundant patterns. This leads to more structured and interpretable image generation.

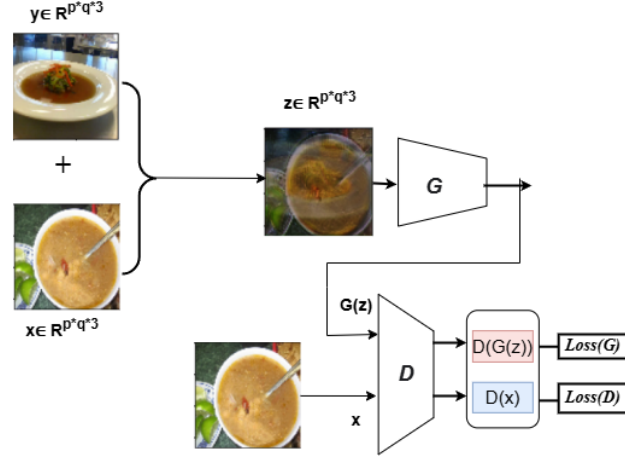
5.5 Step-5: Training Loop

This step includes the full training pipeline for SWGAN. Optimizers such as Adam are initialized for both the generator and discriminator. For each epoch, the model iterates over batches generated from the custom data loader. During every batch update, the generator and discriminator are trained using the `train_step` function. Training losses, including generator loss, discriminator loss, and sparse loss, are logged for analysis. Progress is printed after each epoch to monitor performance and convergence.

5.6 Step-6: Evaluation

After training, SWGAN is evaluated by generating synthetic images using the trained generator. These outputs are visualized to assess their quality, diversity, and realism. In addition, model checkpoints are saved for future inference or fine-tuning. Plots of generator and discriminator loss curves (including sparse and Wasserstein components) are generated to visualize the training dynamics and stability throughout the epochs.

5.7 Model Architecture



$$Loss(G) = Mean(D(G(z))) + 0.001 * (x * \log(x/G(z)) + (1-x) * \log((1-x)/(1-G(z))))$$
$$Loss(D) = Mean(D(G(z))) - Mean(D(x))$$

6 Experiments - Datasets Used

6.1 Libraries Used

- **TensorFlow and Keras:** Frameworks for building and training neural networks, providing tools for designing both the generator and discriminator models in SWGAN.
- **NumPy, Random, and Math:** Libraries for numerical computations, random number generation, and mathematical functions, essential for data preprocessing and augmentation.
- **Cv2 and Matplotlib:** Libraries for image processing and visualization, used for handling input images and visualizing generated samples and training progress.

6.2 MiniImageNet Dataset:



7 Formulas Used :

Wasserstein GAN Losses

The discriminator loss is:

$$\mathcal{L}_D = \mathbb{E}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x})] - \mathbb{E}_{x \sim \mathbb{P}_r} [D(x)]$$

The generator loss is:

$$\mathcal{L}_G = -\mathbb{E}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x})]$$

Sparse Loss

$$\mathcal{L}_{\text{sparse}} = \sum_{j=1}^d \rho \log \left(\frac{\rho}{\hat{\rho}_j} \right) + (1 - \rho) \log \left(\frac{1 - \rho}{1 - \hat{\rho}_j} \right)$$

8 Experiments - Comparative Analysis

Comparison with traditional GANs and WGANs

Evaluation metrics: Mode collapse mitigation and image quality improvement

9 System Hardware

Google Colab TP4 GPU

RAM Used: 7.66 GB/12.67 GB

Disk Used: 29.10 GB/78.19 GB

10 Sample Code

✓ SWGAN

```
[ ] from google.colab import drive
    #drive.mount("/content/gdrive/", force_remount=True)
    drive.mount('/content/gdrive/')
```

↗ Mounted at /content/gdrive/

```
[ ] import os
    root_path = '/content/gdrive/My Drive/Project/Data_miniImageNet'
    print(os.getcwd())
    os.chdir(root_path)
    !ls
```

↗ /content
6_epochs.jpeg mini-imagenet-cache-train.pkl
FSL_Generative_adversarial_encoder.ipynb mini-imagenet-cache-val.pkl
mini-imagenet-cache-test.pkl

✓ Importing Libraries

```
[ ] import tensorflow as tf
    import math
    import matplotlib.pyplot as plt
    import os
    import keras
    import numpy as np
    import numpy.random as rng
    import random
    import tensorflow
    from tensorflow.keras import backend as K
    from tensorflow.keras import layers
    from tensorflow.keras.optimizers import Adam
    from tensorflow.keras.models import Sequential, load_model, Model, save_model
    from tensorflow.keras.layers import *
    from tensorflow.keras import regularizers as rg
    from tensorflow.keras import callbacks as cb
    import cv2
    import numpy as np
    from scipy import signal
    #import tensorflow_probability as tfp
```

✓ Loading Dataset

```
[ ] import pandas as pd
df = pd.read_pickle("mini-imagenet-cache-train.pkl")
x_train=np.array(df['image_data']); y_train = df['class_dict']
#x_train = np.reshape(np.mean(x_train,axis=3),(-1,84,84,1)).astype('float32')
x_train = np.reshape(x_train,(-1,84,84,3)).astype('float32')
x_train /=255 #(x_train1-127)/127
print(x_train.shape, len(y_train))
#.....
df = pd.read_pickle("mini-imagenet-cache-val.pkl")
x_val=df['image_data']; y_val = df['class_dict']
#x_val = np.reshape(np.mean(x_val,axis=3),(-1,84,84,1)).astype('float32')
x_val = np.reshape(x_val,(-1,84,84,3)).astype('float32')
x_val /=255 #(x_val1-127)/127 # 255
print(x_val.shape, len(y_val))
#.....
df = pd.read_pickle("mini-imagenet-cache-test.pkl")
x_test=df['image_data']; y_test = df['class_dict']
#x_test = np.reshape(np.mean(x_test,axis=3),(-1,84,84,1)).astype('float32')
x_test = np.reshape(x_test,(-1,84,84,3)).astype('float32')
x_test /= 255
#x_test=(x_test1-127)/127
print(x_test.shape, len(y_test))
```

✓ Data generation for generator

```
▶ k=0.01
#del LabelTrain, LabelVal, LabelTest,Data_GeneratorRaviLabel1,loadLabel1,Data_GeneratorRaviLabel
#del Data_GeneratorRaviLabel
randLayer=np.random.normal(0.1, 0.05,size=(84,84,3))
idxx=random.sample(list(y_test), 5)

# import cv2
# def flip (img):
#     img=np.reshape(img, (84,84,3))
#     img2= cv2.flip(img, 1)
#     return img2 #np.expand_dims(img2, axis=-1)
# s=5
# def cv2_zoom(img):
#     m,n,p=img.shape
#     cropped_img = img[s:m-s, s:n-s,:]
#     result = cv2.resize(cropped_img, (84, 84))
#     return result

class Data_GeneratorRaviLabel(tf.keras.utils.Sequence):

    #def __init__(self, datatype='train', dim=(32,32), n_channels=1, way=20, shot=1, query=1, num_batch=10):
    def __init__(self, datatype='train', dim=(84,84,1), num_per_class=1, n_channels=1, way = 3,shot=1, num_batch=10):
        'Initialization'
        global k,idxx, mask
        self.data=x_train
        self.y=y_train

        if(datatype=='val'):
            self.data=x_val
            self.y=y_val
```

	<pre> if(datatype=='test'): self.data=x_test self.y=y_test self.dim = dim self.n_channels = n_channels #print(num_per_class) self.num_batch = num_batch self.n_classes = len(self.data) self.on_epoch_end() self.shot = shot self.way = way #self.noise1=np.abs(np.random.normal(0,0.3,(84,84))) self.maskk=np.repeat(np.reshape(mask[:, :, :], (1,84,84,3)),self.way*self.shot, axis=0) def __len__(self): 'Denotes the number of batches per epoch' return self.num_batch def __getitem__(self,index): 'Generate one batch of data' X_query, X_query2 = self.__data_generation() return X_query, X_query2 def on_epoch_end(self): 'Updates indexes after each epoch' pass def __data_generation(self): X_sample=[]; X_query=[]; X_query1=[]; X_query2=[]; idx=random.sample(list(self.y), self.way) label = np.zeros((self.way*self.shot, self.way)) layr1 = random.sample([0,1,2], 1)[0] layr2 = (layr1+1)%3 #random.sample([0,1,2], 1)[0] for i in range(self.way): sample_idx1 = random.sample(self.y[idx[i]], 2*self.shot) </pre>	
	<pre> for s in range(self.shot): imgg = self.data[sample_idx1[s]] noise = np.random.normal(loc=0.1, scale=0.01, size=imgg.shape) imgg_noisy = imgg + noise imgg_noisy = np.clip(imgg_noisy, 0.0, 1.0) X_query.append(imgg_noisy) # X_query.append(imgg) img2 = imgg.copy() img01 = self.data[sample_idx1[self.shot+s]] # # #img2[:, :, layr2] = img01[:, :, layr2] #img2[:, :, layr2]=img2[:, :, layr2]*(1-mask[:, :, layr1])+ img01[:, :, layr2]*(mask[:, :, layr1]+0.1) #img2=img2*(1-mask)+ img01*(mask) # img2=1.5*img2*(1-mask) - 0.5*(img2-img01*(mask)) img2=0.7*img2 + 0.3*img01 #img2=1.5*img2*(1-0.5)# - 0.5*(img2-img01*(mask)) # img2 = img2 * (1 - mask) + img01 * mask * 0.7 + 0.3 * img2 # alpha = 0.6 # Controls how much of img2 contributes outside the mask # beta = 0.8 # Controls how much of img01 contributes inside the mask # gamma = 0.2 # Residual blend factor # img2 = alpha * img2 * (1 - mask) + beta * img01 * mask + gamma * img2 X_query2.append(img2) #Masked # X_query2.append(img01) #support return np.array(X_query), np.array(X_query2) #X_noise, </pre>	

```

train_way = 5
shot = 2
n=600

val_loader = Data_GeneratorRaviLabel(datatype='train', num_per_class=n,way = train_way,shot=shot, num_batch=1)
x,z = val_loader[0]
print(x.shape,z.shape)

img1,img3=val_loader[0]
fig = plt.figure(figsize=(6,10))
p=3
for j in range(5):
    plt.subplot(5,p,p*j+1)
    plt.imshow(img1[j, :, :,:] )
    plt.xlabel('Input')
    # plt.subplot(5,p,p*j+2)
    # plt.imshow(img2[j, :, :,:] )
    plt.subplot(5,p,p*j+3)
    plt.imshow(img3[j, :, :,:] )
    plt.xlabel('Interpolated')
    #plt.axis('off')
plt.show()

```

Generator

```

import tensorflow as tf
from tensorflow.keras import layers, Model, Input

# Assuming you've already defined other functions like make_discriminator_model

def make_generator_model(dim=(84, 84, 3)): # Updated input dimension
    inputs = Input(shape=dim)

    x = layers.Conv2D(64, (3, 3), padding='same')(inputs)
    x = layers.LayerNormalization()(x)
    x = layers.LeakyReLU()(x)

    x = layers.Conv2D(64, (3, 3), padding='same')(x)
    x = layers.LayerNormalization()(x)
    x = layers.LeakyReLU()(x)

    x = layers.Conv2D(64, (3, 3), padding='same')(x)
    x = layers.LayerNormalization()(x)
    x = layers.LeakyReLU()(x)

    hidden = layers.Conv2D(64, (1, 1), padding='same')(x)
    x = layers.LayerNormalization()(hidden)
    x = layers.LeakyReLU()(x)

    # Changed to 3 output channels
    outputs = layers.Conv2D(3, (1, 1), padding='same', use_bias=False, activation='sigmoid')(x)

    return Model(inputs=inputs, outputs=[outputs, hidden])

```

✓ Discriminator

```
from tensorflow.keras import layers, Sequential

def make_discriminator_model():
    model = Sequential()
    model.add(layers.Conv2D(64, (3, 3), padding='valid', input_shape=(84, 84, 3)))
    model.add(layers.LayerNormalization())
    model.add(layers.ReLU())
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(64, (3, 3), padding='same'))
    model.add(layers.LayerNormalization())
    model.add(layers.ReLU())
    model.add(layers.MaxPooling2D(2, 2))

    model.add(layers.Conv2D(64, (3, 3), padding='same'))
    model.add(layers.LayerNormalization())
    model.add(layers.MaxPooling2D(2, 2))
    model.add(layers.ReLU())
    model.add(layers.Dropout(0.1))

    model.add(layers.Conv2D(64, (1, 1), padding='same'))
    model.add(layers.LayerNormalization())
    model.add(layers.MaxPooling2D(2, 2))
    model.add(layers.Activation('relu'))
    model.add(layers.Flatten())

    model.add(layers.Dense(1, activation='sigmoid'))
    return model
```

✓ Training Functions

```
# Optimizers with adjusted learning rates for better stability
generator_optimizer = tf.keras.optimizers.Adam(1e-4, beta_1=0.5, beta_2=0.9)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4, beta_1=0.5, beta_2=0.9)

dim = (84, 84, 3)
generator = make_generator_model(dim)
discriminator = make_discriminator_model()

# Optional Exponential Moving Average (EMA)
ema = tf.train.ExponentialMovingAverage(decay=0.999)

# Loss functions
def wgan_discriminator_loss(real_output, fake_output):
    return tf.reduce_mean(fake_output) - tf.reduce_mean(real_output)

def wgan_generator_loss(fake_output):
    return -tf.reduce_mean(fake_output)

# def sparseLoss(x):
#     target_activations, activations = x
#     rho = tf.reduce_mean(target_activations, axis=1)
#     rho_hat = tf.reduce_mean(activations, axis=1)
#     eps = 1e-10
#     rho_hat = tf.clip_by_value(rho_hat, eps, 1. - eps)
#     kl_div = rho * tf.math.log(rho / rho_hat) + (1 - rho) * tf.math.log((1 - rho) / (1 - rho_hat))
#     return tf.reduce_sum(kl_div)

def sparseLoss(x):
    target_activations, activations = x
    # Convert both target_activations and activations to tf.float32
    target_activations = tf.cast(target_activations, tf.float32)
    activations = tf.cast(activations, tf.float32)
    rho = tf.reduce_mean(target_activations, axis=1)
```

```

    rho_hat = tf.reduce_mean(activations, axis=1)
    eps = 1e-10
    # rho_hat = tf.clip_by_value(rho_hat, eps, 1. - eps)
    rho_hat = tf.clip_by_value(rho_hat, 1e-6, 1-1e-6)
    kl_div = rho * tf.math.log(rho / rho_hat) + (1 - rho) * tf.math.log((1 - rho) / (1 - rho_hat))
    return tf.reduce_sum(kl_div)

#USE_SPARSE_LOSS = False

USE_SPARSE_LOSS = True

# def gradient_penalty(discriminator, real_images, fake_images):
#     batch_size = tf.shape(real_images)[0]
#     alpha = tf.random.uniform([batch_size, 1, 1, 1], 0.0, 1.0)
#     interpolated = alpha * real_images + (1 - alpha) * fake_images

#     with tf.GradientTape() as tape:
#         tape.watch(interpolated)
#         pred = discriminator(interpolated, training=True)

#     grads = tape.gradient(pred, [interpolated])[0]
#     norm = tf.sqrt(tf.reduce_sum(tf.square(grads), axis=[1, 2, 3]) + 1e-10)
#     penalty = tf.reduce_mean((norm - 1.0) ** 2)

#     return penalty

def gradient_penalty(discriminator, real_images, fake_images):
    batch_size = tf.shape(real_images)[0]
    # Explicitly set the dtype of alpha to tf.float32
    alpha = tf.random.uniform([batch_size, 1, 1, 1], 0.0, 1.0, dtype=tf.float32)
    # Cast real_images and fake_images to tf.float32 for consistency
    real_images = tf.cast(real_images, tf.float32)
    fake_images = tf.cast(fake_images, tf.float32)
    interpolated = alpha * real_images + (1 - alpha) * fake_images

def gradient_penalty(discriminator, real_images, fake_images):
    batch_size = tf.shape(real_images)[0]
    # Explicitly set the dtype of alpha to tf.float32
    alpha = tf.random.uniform([batch_size, 1, 1, 1], 0.0, 1.0, dtype=tf.float32)
    # Cast real_images and fake_images to tf.float32 for consistency
    real_images = tf.cast(real_images, tf.float32)
    fake_images = tf.cast(fake_images, tf.float32)
    interpolated = alpha * real_images + (1 - alpha) * fake_images

    with tf.GradientTape() as tape:
        tape.watch(interpolated)
        pred = discriminator(interpolated, training=True)

    grads = tape.gradient(pred, [interpolated])[0]
    norm = tf.sqrt(tf.reduce_sum(tf.square(grads), axis=[1, 2, 3]) + 1e-10) # Add small epsilon for stability
    penalty = tf.reduce_mean((norm - 1.0) ** 2)

    return penalty

# Training steps
@tf.function
def train_discriminator_step(x, z, lambda_gp=10.0):
    with tf.GradientTape() as disc_tape:
        fake_images, _ = generator(z, training=True)
        real_output = discriminator(x, training=True)
        fake_output = discriminator(fake_images, training=True)

        gp = gradient_penalty(discriminator, x, fake_images)
        disc_loss = wgan_discriminator_loss(real_output, fake_output) + lambda_gp * gp

    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))
    return disc_loss

```

```

@tf.function
def train_generator_step(x, z):

    with tf.GradientTape() as gen_tape:
        fake_images, _ = generator(z, training=True)
        fake_output = discriminator(fake_images, training=True)

        gen_loss = wgan_generator_loss(fake_output)

        if USE_SPARSE_LOSS:
            sparse_weight = 0.001
            gen_loss += sparse_weight * sparseLoss([x, fake_images])

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))

    # Apply EMA
    # ema.apply(generator.trainable_variables)

    return gen_loss # Always return gen_loss, even if USE_SPARSE_LOSS is True

```

```

num_batch = 100 # or set appropriately

def train(epochs):
    global KLD_cmdgLoss1, KLD_cmddLoss1

    # Reinitialize ema within the training loop
    # This ensures it's correctly created each time train is called
    ema = tf.train.ExponentialMovingAverage(decay=0.999)

    for epoch in range(epochs):
        dataset = Data_GeneratorRaviLabel(datatype='train', num_per_class=n, way=train_way, shot=shot, num_batch=num_batch)
        epoch_gen_loss, epoch_disc_loss = 0.0, 0.0

        for j in range(len(dataset)):
            image_batch1, image_batch2 = dataset[j]

            # Perform 5 discriminator updates for each generator update
            for _ in range(5):
                disc_loss = train_discriminator_step(image_batch1, image_batch2)
                epoch_disc_loss += disc_loss
            gen_loss = train_generator_step(image_batch1, image_batch2)
            epoch_gen_loss += gen_loss

        avg_gen_loss = epoch_gen_loss / num_batch
        avg_disc_loss = epoch_disc_loss / (num_batch * 5) # 5 discriminator steps

        print(f"[Epoch {epoch+1}/{epochs}] Generator Loss: {avg_gen_loss.numpy():.4f}, Discriminator Loss: {avg_disc_loss.numpy():.4f}")

        KLD_cmdgLoss1.append(avg_gen_loss.numpy())
        KLD_cmddLoss1.append(avg_disc_loss.numpy())

    # Initialize
    EPOCHS = 75
    KLD_cmdgLoss1 = []
    KLD_cmddLoss1 = []

    # Train
    train(EPOCHS)

```

✓ Save and Load Functions

```
import pickle
def saveData(data,file_name):
    open_file = open(file_name, "wb")
    pickle.dump(data, open_file)
    open_file.close()

def loadData(file_name):
    open_file = open(file_name, "rb")
    hist = pickle.load(open_file)
    open_file.close()
    return hist
```

✓ Changing output batch

```
[ ] import pickle

# Extract a batch from the dataset
dataset = Data_GeneratorRaviLabel(way=train_way, datatype='train', shot=shot, num_batch=1)
(image1,image2, image3) = dataset[0]
```

✓ Display

```
# Access the image output from the generator (first element of the list)
# noise=tf.concat((image1, image3), axis=-1)
# noise = noise[:, :, :3]
noise = image3
predictions = generator(noise, training=False)
image_output = predictions[0]

fig= plt.figure(figsize=(10,20))
k=10;p=4
for i in range(k):
    plt.subplot(k, p, p*i+1)
    plt.imshow(image1[i, :, :,:])
    #plt.axis('off')
    # plt.xlabel('Input')
    plt.xlabel('Input')

    plt.subplot(k, p, p*i+2)
    plt.imshow(image2[i, :, :,:])
    #plt.axis('off')
    plt.xlabel('Complementary')

    plt.subplot(k, p, p*i+3)
    plt.imshow(image3[i, :, :,:])
    #plt.axis('off')
    # plt.xlabel('support')
    plt.xlabel('Interpolated')

    plt.subplot(k, p, p*i+4)
    # Now you can access the image output correctly
    plt.imshow(image_output[i, :, :,:], cmap='gray')
    #plt.axis('off')
    plt.xlabel('Generated')

plt.savefig('image_at_epoch_{:04d}.png'.format(epoch))
plt.show()
```


✓ Save generator

```
[ ] generator.save('/content/gdrive/My Drive/Project/KLD_cmd_gan_generator.keras')
```

✓ Load generator

```
▶ generator = tf.keras.models.load_model('/content/gdrive/My Drive/Project/KLD_cmd_gan_generator.keras')
```

✓ Saving for Graphs

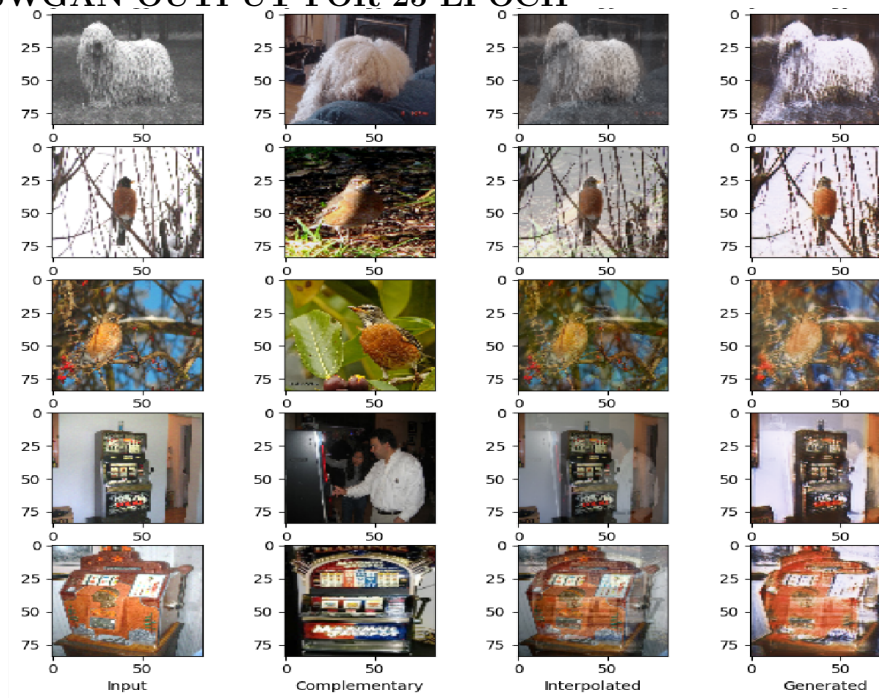
```
[ ] saveData(KLD_cmdgLoss1, '/content/gdrive/My Drive/Project/KLD_cmdgLoss11.npy' )  
    saveData(KLD_cmddLoss1, '/content/gdrive/My Drive/Project/KLD_cmddLoss11.npy' )
```

✓ Plotting

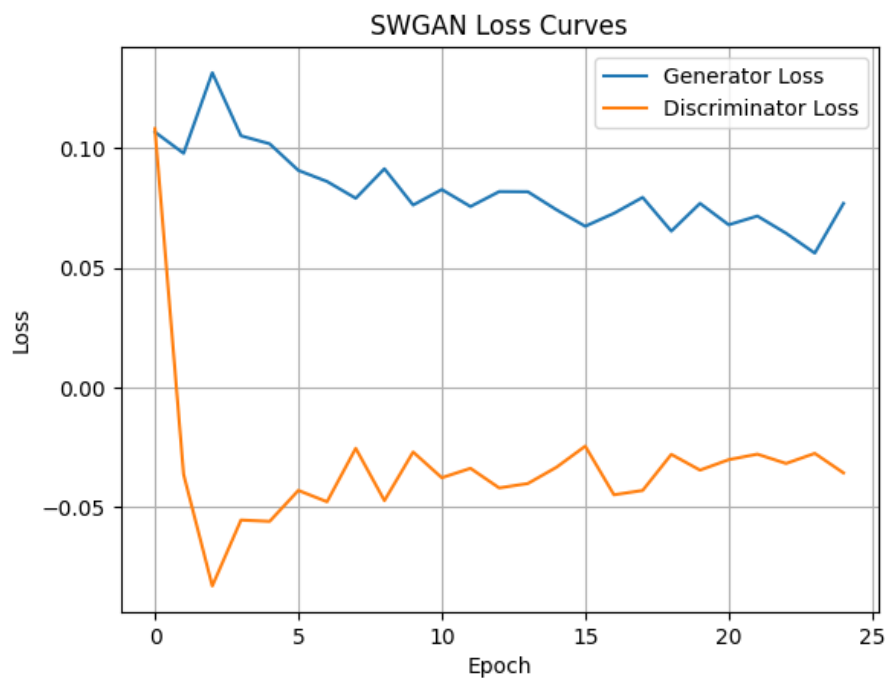
```
▶ KLD_cmdd1=loadData('/content/gdrive/My Drive/Project/KLD_cmddLoss11.npy')  
  KLD_cmdg1=loadData('/content/gdrive/My Drive/Project/KLD_cmdgLoss11.npy')  
  plt.plot(KLD_cmdd1)  
  plt.plot(KLD_cmdg1)  
  plt.legend(["WGAN Discriminator loss", "WGAN Generator loss"])  
  plt.show()
```

11 Output

SWGAN OUTPUT FOR 25 EPOCH



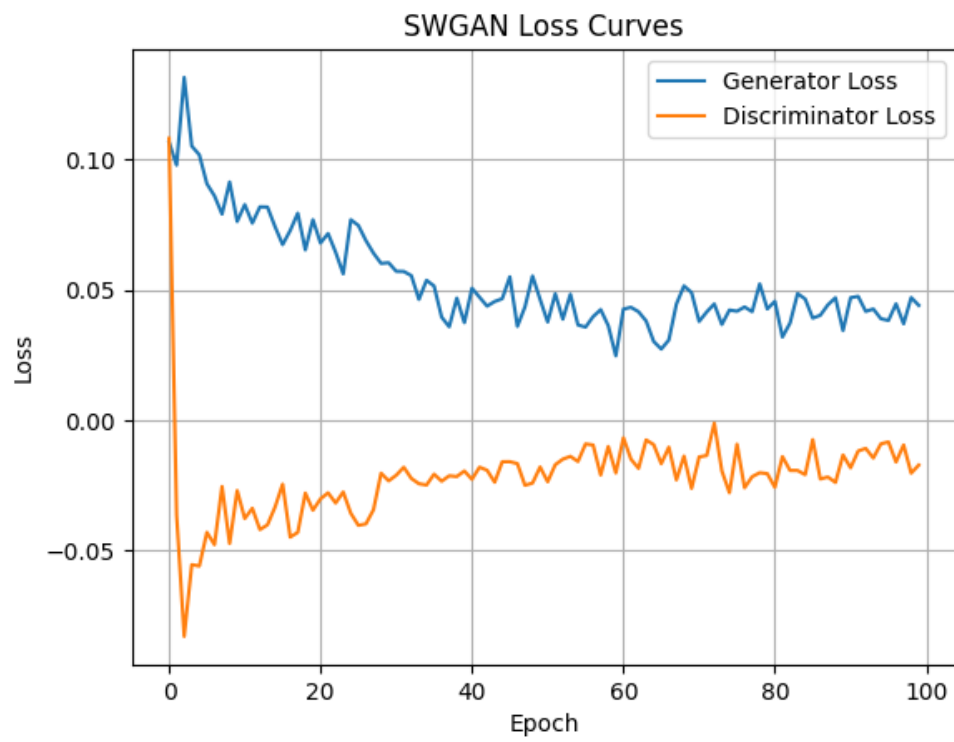
SWGAN GRAPH FOR 25 EPOCHS



12 SWGAN OUTPUT FOR 100 EPOCH



13 SWGAN GRAPH FOR 100 EPOCHS



14 Conclusion

14.1 Summary of SWGAN’s Contributions

The SWGAN (Sparse Wasserstein Generative Adversarial Network) marks a notable advancement in the domain of generative adversarial networks. Designed to overcome persistent challenges such as mode collapse and unstable training dynamics, SWGAN integrates the strengths of Wasserstein GANs with an additional sparse loss regularization. This combination promotes the learning of structured, disentangled, and diverse representations, leading to more stable training and improved output diversity. By enforcing sparsity in feature activations, SWGAN enhances the generator’s ability to capture essential data characteristics, resulting in high-quality and realistic synthetic images. The framework demonstrates improved convergence behavior and robustness, making it a powerful tool for generative modeling and a promising foundation for future research.

14.2 Advancements over Traditional GANs and WGANs

CMD-GAN builds upon the foundational principles of traditional GANs and Wasserstein GANs (WGANs), introducing several key improvements:

- **Enhanced Training Stability**
- **Mitigation of Mode Collapse**
- **Improved Performance Metrics**

14.3 Future Potential in Generative Modeling and Data Augmentation

The advancements introduced by SWGAN open up exciting possibilities for future research and applications in generative modeling and data augmentation:

- **Generative Modeling**
- **Data Augmentation**
- **Medical Imaging**

15 Future Enhancements

15.1 Areas for Future Research and Development

The SWGAN presents a robust and promising framework for generative modeling by integrating sparse regularization with Wasserstein-based adversarial training. However, several areas offer scope for further research and development to enhance its effectiveness:

- **Integration of Advanced Sparsity Mechanisms:** While SWGAN utilizes a basic sparse loss for regularization, future work could explore more sophisticated sparsity-enforcing techniques such as dynamic sparsity, group sparsity, or biologically inspired sparse coding. These could improve feature disentanglement and enhance generalization.
- **Refinement of Disentangled Representations:** One of SWGAN’s strengths lies in learning structured and interpretable features. Research can be directed towards improving disentanglement by modifying the architecture or introducing new loss functions that better isolate semantic factors in the data.
- **Improved Training Stability and Convergence:** Although SWGAN leverages Wasserstein loss and gradient penalty for stability, there is room to enhance convergence speed and reduce sensitivity to hyperparameters. Exploring adaptive gradient clipping, normalization techniques like spectral normalization, or hybrid optimization algorithms could yield further improvements.

15.2 Potential Applications and Extensions of SWGAN

The architecture and regularization strategies of SWGAN open up a wide range of applications and future extensions across multiple domains:

- **Realistic Image Generation and Editing:** SWGAN can be applied to generate high-quality, diverse synthetic images with fine structural detail. Its sparse feature learning is particularly suited for image manipulation tasks such as texture synthesis, object transfer, and guided inpainting.
- **Efficient Data Augmentation for Deep Learning:** By generating diverse and well-structured synthetic samples, SWGAN can effectively augment limited datasets. This can boost model performance in classification, detection, and segmentation tasks, particularly in low-data regimes like few-shot learning.
- **Medical and Scientific Imaging:** The ability of SWGAN to learn disentangled and sparse representations makes it ideal for medical imaging applications, such as generating varied but realistic pathology images, enhancing diagnostic images, and synthesizing rare disease cases to balance datasets.
- **Anomaly and Outlier Detection:** SWGAN’s structured feature learning allows it to model the normal data distribution effectively. This capability can be harnessed for unsupervised anomaly detection in areas such as cybersecurity, fraud detection, industrial quality control, and healthcare monitoring.

References

- [1] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative adversarial networks: An overview," *IEEE signal processing magazine*, vol. 35, no. 1, pp. 53-65, 2018.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, et al., "Generative adversarial nets," in *Advances in neural information processing systems*, pp. 2672-2680, 2014.
- [3] M. Zhang, B. Du, L. Zhang, T. Chen, and Y. Wu, "Unsupervised feature extraction in hyperspectral images based on Wasserstein generative adversarial network," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 57, no. 5, pp. 2669-2688, 2018.
- [4] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE transactions on pattern analysis and machine intelligence*, vol. 39, no. 12, pp. 2481, 2017.
- [5] T.-A. Song, S. Chowdhury, K. Kim, K. Gong, G. El Fakhri, Q. Li, et al., "Super-resolution PET using a very deep convolutional neural network," in *Proc IEEE Nucl Sci Symp Med Imag Conf*, 2018.